

# ADICIONAL PILAS Y COLAS

- 1) Implemente la clase Pila con la siguiente especificación:

|                                |                                                                                                      |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <code>push(elemento)</code>    | Agrega un elemento a la pila.                                                                        |
| <code>pop()</code>             | Extraer un elemento de pila.                                                                         |
| <code>isEmpty()</code>         | Retorna true si pila está vacía.                                                                     |
| <code>top()</code>             | Retorna el elemento al tope de la pila sin sacarlo.                                                  |
| <code>size()</code>            | Retorna la cantidad de elementos que tiene pila.                                                     |
| <code>reverse()</code>         | Retorna una nueva pila con los elementos en posición invertida (el primero en último lugar).         |
| <code>pushAll(otrapila)</code> | Agrega a la pila receptora todos los elementos de otrapila (respetando el orden original de salida). |

- 2) Implemente la clase Cola con la siguiente especificación:

|                                |                                                                                                      |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <code>push(elemento)</code>    | Agrega un elemento a la cola.                                                                        |
| <code>pop()</code>             | Extraer un elemento de cola.                                                                         |
| <code>isEmpty()</code>         | Retorna true si cola está vacía.                                                                     |
| <code>top()</code>             | Retorna el elemento al tope de la cola sin sacarlo.                                                  |
| <code>size()</code>            | Retorna la cantidad de elementos que tiene cola.                                                     |
| <code>reverse()</code>         | Retorna una nueva cola con los elementos en posición invertida (el primero en último lugar).         |
| <code>pushAll(otracola)</code> | Agrega a la cola receptora todos los elementos de otracola (respetando el orden original de salida). |

- 3) Escriba una función que reciba una string y devuelve el string invertido utilizando una Pila.
- 4) Escriba un función que reciba un string y devuelva true si la cadena esta balanceada o false en caso contrario.

Una cadena de caracteres S está balanceada si tiene alguna de las siguientes formas: S = "" (string de longitud cero)

S = "(T)"

S = "[T]"

S = "{T}"

S = "TU"

Donde T y U son cadenas balanceadas.

Por ejemplo, “{ [ ( ) ( ) ] { [ ] } }” está balanceada, pero “ ( [ ] )” no lo está.

¿Qué colección de datos (Pila o Cola) utilizaría para resolver este problema?

- 5) Utilizando la clase Cola, escriba una función llamada que reciba 2 Colas con números enteros y devuelva una nueva Cola con sus elementos sumados uno a uno.

Ej:

| Cola A | Cola B | Cola Resultado |
|--------|--------|----------------|
| 3      | 6      | 9              |
| 4      | 2      | 6              |
| 2      | 9      | 11             |
| 8      | 11     | 19             |
| 12     | 3      | 15             |

- 6) Implemente utilizando la clase Pila para la resolución del juego de las Torres de Hanoi para tres discos.
- 7) Intente generalizar el juego de las Torres de Hanoi para  $n$  discos.
- 8) Implemente un TAD (tipo de datos abstracto) Conjunto con la siguiente especificación:
- `agregar(elemento)` Agrega un elemento al conjunto.
  - `pertenece(elemento)` Retorna true si el elemento está en el conjunto.
  - `esVacio()` Retorna true si el conjunto está vacío.
  - `cardinalidad()` Devuelve la cantidad de elementos del conjunto
  - `union(conj)` Retorna un nuevo conjunto con la unión del conjunto receptor y conj.
  - `interseccion(conj)` Retorna un nuevo conjunto con la interseccion del conjunto receptor y conj.
  - `diferencia(conj)` Retorna un nuevo conjunto con la diferencia entre el conjunto receptor y conj.
  - `incluye(conj)` Devuelve true si el conjunto receptor incluye a conj.
  - `incluido(conj)` Devuelve true si el conjunto receptor está incluido en conj.

EXTRA:

Implemente una aplicación que permita manejar colas de servicios para un banco. Un cliente llega y según lo que necesite le corresponde un número dentro de una de las colas. El personal de atención va llamando a cada cliente eligiendo la cola que desea atender. Este sistema deberá permitir simular la llegada de un cliente ingresando por consola la letra C seguida de un número de servicio. El sistema deberá entregar un número de atención (el que corresponda según la cola actual del servicio).

También deberá permitir atender a un cliente ingresando por consola la letra A seguida del número de servicio. El sistema deberá mostrar en consola el número que se está llamando.

Utilice la clase Cola para la resolución de este sistema.